15

IMPROVING YOUR ASTROPHOTOGRAPHY WITH PLANET STACKING

If you've ever looked through a telescope at Jupiter, Mars, or Saturn, you were probably a little disappointed. The planets appeared small and featureless. You wanted to zoom in and crank up the magnification, but it didn't work. Anything bigger than about 200x magnification tends to be blurry.

The problem is air turbulence, or what astronomers call *seeing*. Even on a clear night, the air is constantly in motion, with thermal updrafts and downdrafts that can easily blur the pinpoints of light that represent celestial objects. But with the commercialization of the *charge-coupled device (CCD)* in the 1980s, astronomers found a way to overcome the turbulence. Digital photography permits a technique known as *image stacking*, in which many photos—some good, some bad—are averaged together, or stacked, into a single image. With enough photos, the persistent, unchanging features (like a planet's surface) dominate transient features (like a stray cloud). This allows astrophotographers to increase magnification limits, as well as compensate for less-than-optimal seeing conditions.

In this chapter, you'll use a third-party Python module called `pillow` to stack hundreds of images of Jupiter. The result will be a single image with

a higher signal-to-noise ratio than any of the individual frames. You'll also work with files in different folders than your Python code and manipulate both the files and folders using the Python operating system (`os`) and shell utilities (`shutil`) modules.

Project #23: Stacking Jupiter

Large, bright, and colorful, the gas giant Jupiter is a favorite target of astrophotographers. Even amateur telescopes can resolve its orange striping, caused by linear cloud bands, and the Great Red Spot, an oval-shaped storm so large it could swallow the Earth (see **Figure 15-1**).

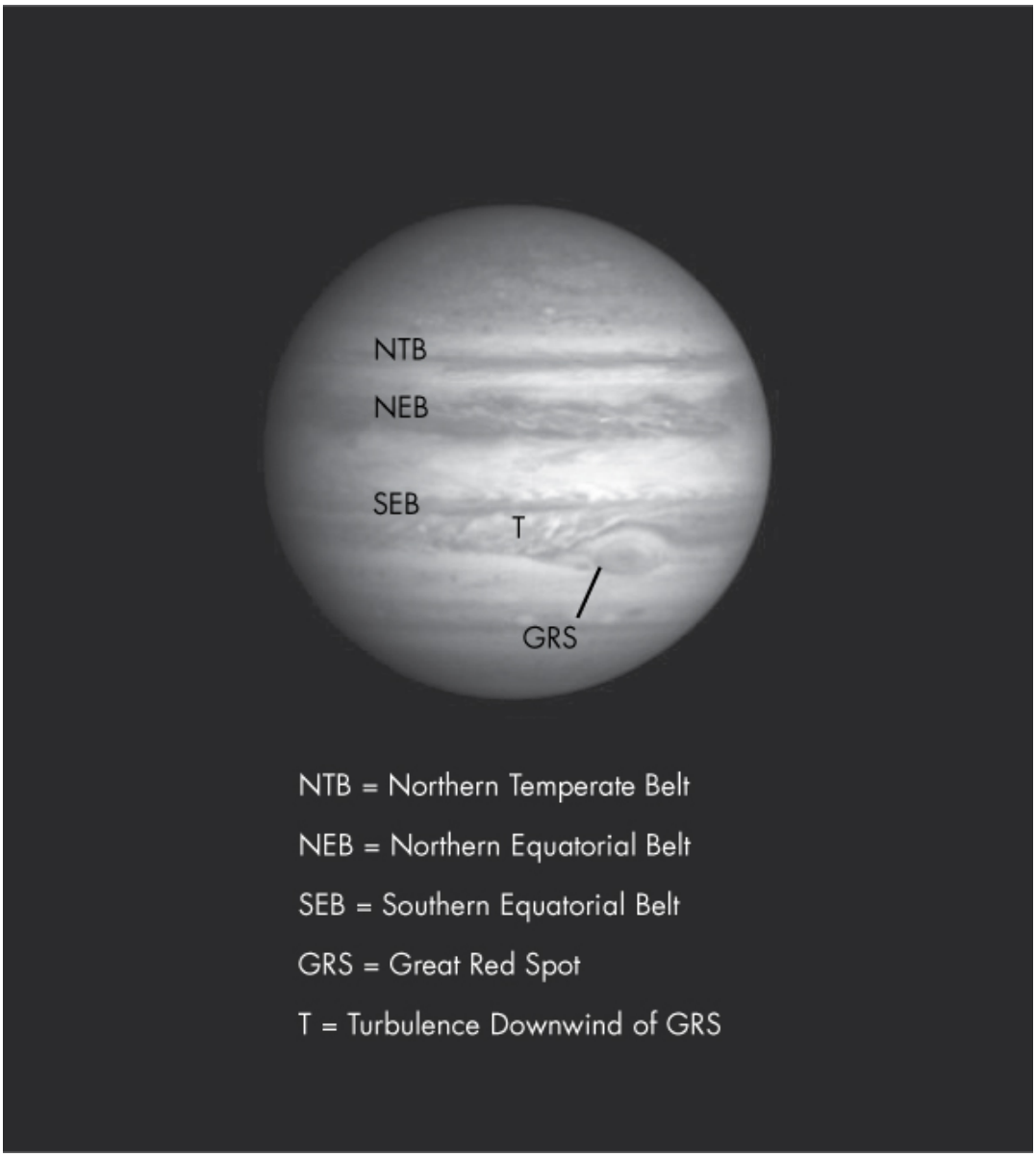

Figure 15-1: Jupiter as photographed by the Cassini spacecraft

Jupiter is a great subject for studying image stacking. Its linear cloud bands and Great Red Spot provide the eye with calibration points for

judging improvements in edge definition and clarity, and its relatively large size makes it easy to detect noise.

Noise manifests itself as "graininess." Each color band has its own artifacts, resulting in colored speckles across an image. The main sources of noise are the camera (electronic readout noise and thermal signal) and photon noise from the light itself, as a variable number of photons strike the sensor over time. Noise artifacts are fortunately random in nature and can be largely canceled out by stacking images.

THE OBJECTIVE

Write programs that crop, scale, stack, and enhance images to create a clearer photograph of Jupiter.

The pillow Module

To work with images, you'll need a free third-party Python module called `pillow`. It's the successor project to the Python Imaging Library (PIL), which was discontinued in 2011. The `pillow` module "forked" the PIL repository and upgraded the code for Python 3.

You can use `pillow` on Windows, macOS, and Linux, and it supports many image formats including PNG, JPEG, GIF, BMP, and TIFF. It offers standard image-manipulation procedures, such as altering individual pixels, masking, handling transparency, filtering and enhancing, and adding text. But the real strength of `pillow` is its ability to edit many images with ease.

Installing `pillow` is easy with the pip tool (for more about pip, see "**Manipulating Word Documents with `python-docx`**" on **page 110**). From the command line, enter **`pip install pillow`**.

Most major Linux distributions include `pillow` in packages that previously contained PIL, so you may already have `pillow` on your system. Regardless of your platform, if PIL is already installed, you'll need to uninstall it before installing `pillow`. For more installation instructions, see **http://pillow.readthedocs.io/en/latest/installation.html**.

Working with Files and Folders

In all the previous projects in this book, you've kept supporting files and modules in the same folder as your Python code. This was handy for simple projects, but not very realistic for broad use, and it's certainly not good when you're dealing with the hundreds of image files you'll generate in this project. Fortunately, Python ships with several modules that can help with this, like `os` and `shutil`. But first, I'll briefly discuss directory paths.

Directory Paths

The directory path is the address to a file or folder. It starts with a root directory, which is designated with a letter (such as *C:*|) in Windows, and a forward slash (*/*) in Unix-based systems. Additional drives in Windows are assigned a different letter than *C*, those in macOS are placed under */volume*, and those in Unix under */mnt* (for "mount").

NOTE

I use the Windows operating system for the examples in this chapter, but you can achieve the same result on macOS and other systems. And as is commonly done, I use the terms directory *and* folder *interchangeably here.*

Pathnames appear differently depending on the operating system. Windows separates folders with a backslash (|), while macOS and Unix systems use a forward slash (*/*). Also, in Unix, folder and file names are case sensitive.

If you're writing your program in Windows and type in pathnames with backslashes, other platforms won't recognize the paths. Fortunately, the `os.path.join()` method will automatically ensure your pathname is suitable for whatever operating system Python is running on. Let's take a look at this, and other examples, in **Listing 15-1**.

```
❶ >>> import os
❷ >>> os.getcwd()
```

```
    'C:\\Python35\\Lib\\idlelib'
❸ >>> os.chdir('C:\\Python35\\Python 3 Stuff')
    >>> os.getcwd()
    'C:\\Python35\\Python 3 Stuff'
❹ >>> os.chdir(r'C:\Python35\Python 3 Stuff\Planet Stacking')
    >>> os.getcwd()
❺ 'C:\\Python35\\Python 3 Stuff\\Planet Stacking'
❻ >>> os.path.join('Planet Stacking', 'stack_8', '8file262.jpg')
    'Planet Stacking\\stack_8\\8file262.jpg'
❼ >>> os.path.normpath('C:/Python35/Python 3 Stuff')
    'C:\\Python35\\Python 3 Stuff'
❽ >>> os.chdir('C:/Python35')
    >>> os.getcwd()
    'C:\\Python35'
```

Listing 15-1: Working with Windows pathnames using the `os` module

After importing the `os` module for access to operating system–dependent functionality ❶, get the *current working directory*, or *cwd* ❷. The cwd is assigned to a process when it starts up; that is, when you run a script from your shell, the cwd of the shell and the script will be the same. For a Python program, the cwd is the folder that contains the program. When you get the cwd, you're shown the full path. Note that you must use extra backslashes in order to escape the backslash characters used as file separators.

Next, you change the cwd using the `os.chdir()` method ❸, passing it the full path in quotes, using double backslashes. Then you get the cwd again to see the new path.

If you don't want to type the double backslash, you can enter an `r` before the pathname argument string to convert it to a *raw string* ❹. Raw strings use different rules for backslash escape sequences, but even a raw string can't end in a single backslash. The path will still be displayed with double backslashes ❺.

If you want your program to be compatible with all operating systems, use the `os.path.join()` method and pass it the folder names and filenames

without a separator character ❻. The `os.path` methods are aware of the system you're using and return the proper separators. This allows for platform-independent manipulation of filenames and folder names.

The `os.path.normpath()` method corrects separators for the system you are using ❼. In the Windows example shown, incorrect Unix-type separators are replaced with backslashes. Native Windows also supports use of the forward slash and will automatically make the conversion ❽.

The full directory path—from the root down—is called the *absolute path*. You can use shortcuts, called *relative paths*, to make working with directories easier. Relative paths are interpreted from the perspective of the current working directory. Whereas absolute paths start with a forward slash or drive label, relative paths do not. In the following code snippet, you can change directories without entering an absolute path—Python is aware of the new location because it is *within* the cwd. Behind the scenes, the relative path is joined to the path leading to the cwd to make a complete absolute path.

```
>>> os.getcwd()
'C:\\Python35\\Python 3 Stuff'
>>> os.chdir('Planet Stacking')
>>> os.getcwd()
'C:\\Python35\\Python 3 Stuff\\Planet Stacking'
```

You can identify folders and save more typing with dot (`.`) and dot-dot (`..`). For example, in Windows, `.\` refers to the cwd, and `..\` refers to the parent directory that holds the cwd. You can also use a dot to get the absolute path to your cwd:

```
>>> os.path.abspath('.')
'C:\\Python35\\Python 3 Stuff\\Planet Stacking\\for_book'
```

Dot folders can be used in Windows, macOS, and Linux. For more on the `os` module, see **https://docs.python.org/3/library/os.html**.

The Shell Utilities Module

The shell utilities module, `shutil`, provides high-level functions for working with files and folders, such as copying, moving, and deleting. Since it's part of the Python standard library, you can load `shutil` simply by typing `import shutil`. You'll see example uses for the module in this chapter's code sections. Meanwhile, you can find the module's documentation at **https://docs.python.org/3.7/library/shutil.html**.

The Video

Brooks Clark recorded the color video of Jupiter used in this project on a windy night in Houston, Texas. It consists of a 101 MB *.mov* file with a runtime of about 16 seconds.

The length of the video is intentionally short. Jupiter's rotation period is about 10 hours, which means still photos may blur with exposure times of only a minute, and features you want to reinforce through stacking video frames can change positions, greatly complicating the process.

To convert the video frames to individual images, I used Free Studio, a freeware set of multimedia programs developed by DVDVideoSoft. The Free Video to JPG Converter tool permits the capture of images at constant time or frame intervals. I set the interval to sample frames across the full length of the video, to improve the odds of capturing some images when the air was still and the seeing good.

A few hundred images should be enough for stacking to show demonstrable improvement. In this case, I captured 256 frames.

You can find the folder of images, named *video_frames*, online with the book's resources at **https://www.nostarch.com/impracticalpython/**. Download this folder and retain the name.

An example frame from the video, in grayscale, is shown in **Figure 15-2**. Jupiter's cloud bands are faint and fuzzy, the Great Red Spot isn't apparent, and the image suffers from low contrast, a common side effect of magnification. Noise artifacts also give Jupiter a grainy appearance.

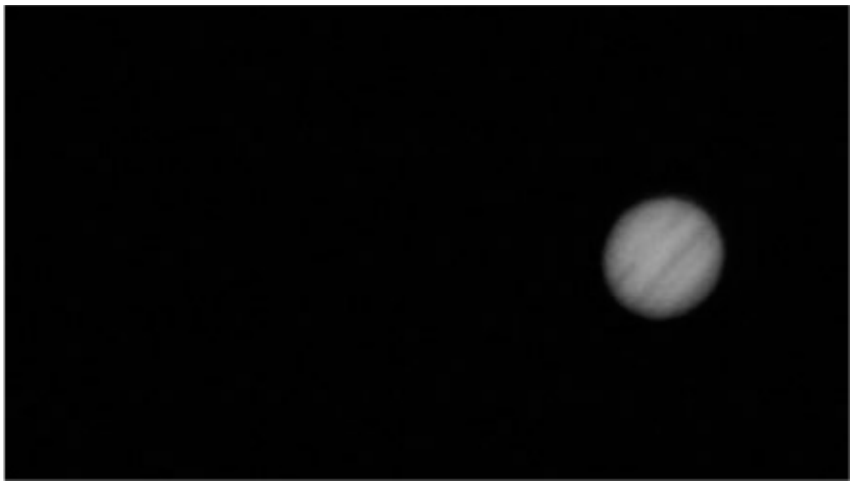

Figure 15-2: Example frame from the video of Jupiter

In addition to those issues, the wind shook the camera, and imprecise tracking caused the planet to drift laterally to the left-hand side of the frame. You can see an example of *lateral drift* in **Figure 15-3**, in which I have overlaid five randomly chosen frames with the black background set to transparent.

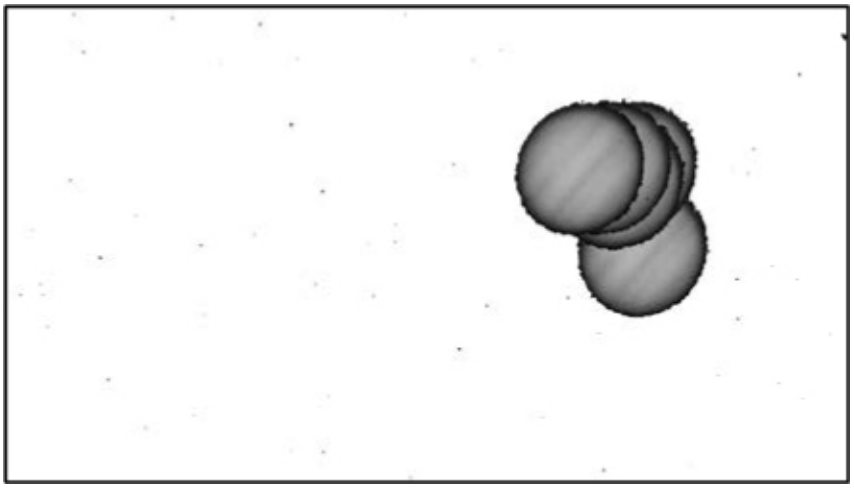

Figure 15-3: An example of shake and drift in the Jupiter video based on five randomly chosen frames

Movement isn't necessarily a bad thing, because shifting the image around can smooth defects associated with the CCD sensor surface, dust on the lens or sensor, and so on. But the key assumption in image stacking is that the images perfectly align so that persistent features, like Jupiter's cloud bands, reinforce each other as you average the images. For the signal-to-noise ratio to be high, the images must be registered.

Image registration is the process of transforming data to the same coordinate system so that it can be compared and integrated. Registration is ar-

guably the hardest part of image stacking. Astronomers typically use commercial software—such as RegiStax, RegiStar, Deep Sky Stacker, or CCDStack—to help them align and stack their astrophotos. You'll get your hands dirty, however, and do this yourself using Python.

The Strategy

The steps required to stack the images are as follows (the first one has already been completed):

1. Extract images from video recording.
2. Crop images around Jupiter.
3. Scale cropped images to the same size.
4. Stack images into a single image.
5. Enhance and filter the final image.

The Code

You can incorporate all the steps into one program, but I chose to distribute them across three programs. This is because you generally want to stop and check results along the way, plus you may want to run later processes, such as enhancement, without having to completely rerun the whole workflow. The first program will crop and scale the images, the second will stack them, and the third will enhance them.

The Cropping and Scaling Code

First, you need to register the images. For large, bright objects like the moon and Jupiter, one approach in astrophotography is to crop each image so that its four borders are tangent with the surface of the body. This removes most of the sky and mitigates any shake and drift issues. Scaling the cropped images will ensure they are all the same size and will smooth them slightly to reduce noise.

You can download *crop_n_scale_images.py* from **https://www.nostarch.com/impracticalpython/**. Keep it in the directory that holds the folder of captured video frames.

Importing Modules and Defining the main() Function

Listing 15-2 performs imports and defines the `main()` function that runs the *crop_n_scale_images.py* program.

crop_n_scale_images.py, part 1

```
❶ import os
   import sys
❷ import shutil
❸ from PIL import Image, ImageOps

   def main():
       """Get starting folder, copy folder, run crop function, & clean folder."'
       # get name of folder in cwd with original video images
    ❹ frames_folder = 'video_frames'

       # prepare files & folders
    ❺ del_folders('cropped')
    ❻ shutil.copytree(frames_folder, 'cropped')

       # run cropping function
       print("start cropping and scaling...")
    ❼ os.chdir('cropped')
       crop_images()
    ❽ clean_folder(prefix_to_save='cropped')  # delete uncropped originals

       print("Done! \n")
```

Listing 15-2: Imports modules and defines the `main()` function

Start by importing both the operating system (`os`) and system (`sys`) ❶. The `os` import already includes an import of `sys`, but this feature may go away in the future, so it's best to manually import `sys` yourself. The `shutil` module contains the shell utilities described earlier ❷. From the imaging library, you'll use `Image` to load, crop, convert, and filter images; you'll also use `ImageOps` to scale images ❸. Note you must use PIL, not `pillow`, in the `import` statement.

Start the `main()` function by assigning the name of the starting folder to the `frames_folder` variable ❹. This folder contains all the original images captured from the video.

You'll store the cropped images in a new folder named *cropped*, but the shell utilities won't create this folder if it already exists, so call the `del_folders()` function that you'll write in a moment ❺. As written, this function won't throw an error if the folder doesn't exist, so it can be run safely at any time.

You should always work off a copy of original images, so use the `shutil.copytree()` method to copy the folder containing the originals to a new folder named *cropped* ❻. Now, switch to this folder ❼ and call the `crop_images()` function, which will crop and scale the images. Follow this with the `clean_folder()` function, which removes the original video frames that were copied into the *cropped* folder and are still hanging around ❽. Note that you use the parameter name when you pass the argument to the `clean_folder()` function, since this makes the purpose of the function more obvious.

Print `Done!` to let the user know when the program is finished.

Deleting and Cleaning Folders

Listing 15-3 defines helper functions to delete files and folders in *crop_n_scale_images.py*. The `shutil` module will refuse to make a new folder if one with the same name already exists in the target directory. If you want to run the program more than once, you first have to remove or rename existing folders. The program will also rename images once they have been cropped, and you'll want to delete the original images before you start stacking them. Since there will be hundreds of image files, these functions will automate an otherwise laborious task.

crop_n_scale_images.py, part 2

```
❶ def del_folders(name):
      """If a folder with a named prefix exists in directory, delete it."""
   ❷ contents = os.listdir()
```

```
    ❸ for item in contents:
        ❹ if os.path.isdir(item) and item.startswith(name):
            ❺ shutil.rmtree(item)

❻ def clean_folder(prefix_to_save):
    """Delete all files in folder except those with a named prefix."""
    ❼ files = os.listdir()
    for file in files:
        ❽ if not file.startswith(prefix_to_save):
            ❾ os.remove(file)
```

Listing 15-3: Defines functions to delete folders and files

Define a function called `del_folders()` for deleting folders ❶. The only argument will be the name of a folder you want to remove.

Next, list the contents of the folder ❷, then start looping through the contents ❸. If the function encounters an item that starts with the folder name and is also a directory ❹, use `shutil.rmtree()` to delete the folder ❺. As you'll see in a moment, a different method is used to delete a folder than to delete a file.

NOTE

Always be careful when using the `rmtree()` method, as it permanently *deletes folders and their contents. You can wipe much of your system, lose important documents unrelated to Python projects, and break your computer!*

Now, define a helper function to "clean" a folder and pass it the name of files *that you don't want to delete* ❻. This is a little counterintuitive at first, but since you only want to keep the last batch of images you've processed, you don't have to worry about explicitly listing any other files in the folder. If the files don't start with the prefix you provide, such as *cropped*, then they are automatically removed.

The process is similar to the last function. List the contents of the folder ❼ and start looping through the list. If the file doesn't start with the prefix you provided ❽, use `os.remove()` to delete it ❾.

Cropping, Scaling, and Saving Images

Listing 15-4 registers the frames captured from the video by fitting a box around Jupiter and cropping the image to the box (**Figure 15-4**). This technique works well with bright images on a field of black (see "**Further Reading**" on **page 343** for another example).

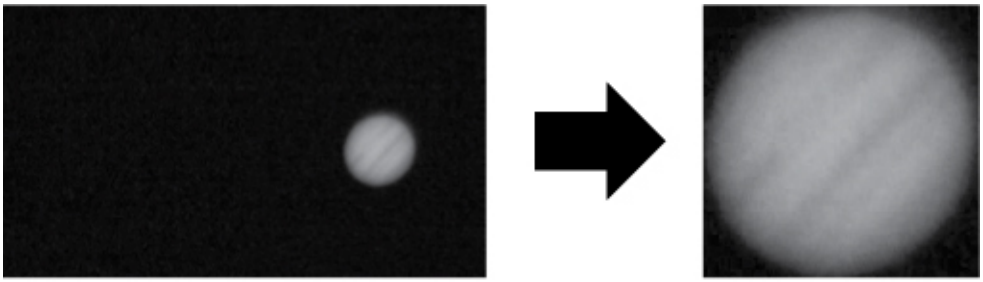

Figure 15-4: Cropping the original video frame to Jupiter to align the images

By cropping the images tightly around Jupiter, you resolve all of the drift and shake issues.

Each cropped image is also scaled to a larger and consistent size and smoothed slightly to reduce noise. The cropped and scaled images will be kept in their own folder, which the `main()` function creates, later.

crop_n_scale_images.py, part 3

```
❶ def crop_images():
      """Crop and scale images of a planet to box around planet."""
   ❷ files = os.listdir()
   ❸ for file_num, file in enumerate(files, start=1):
      ❹ with Image.open(file) as img:
         ❺ gray = img.convert('L')
         ❻ bw = gray.point(lambda x: 0 if x < 90 else 255)
         ❼ box = bw.getbbox()
            padded_box = (box[0]-20, box[1]-20, box[2]+20, box[3]+20)
         ❽ cropped = img.crop(padded_box)
```

```
                scaled = ImageOps.fit(cropped, (860, 860),
                                      Image.LANCZOS, 0, (0.5, 0.5))
                file_name = 'cropped_{}.jpg'.format(file_num)
            ❾ scaled.save(file_name, "JPEG")

    if __name__ == '__main__':
        main()
```

Listing 15-4: Crops initial video frames to a box around Jupiter and rescales

The `crop_images()` function takes no argument ❶ but will ultimately work on a copy—named *cropped*—of the folder containing the original video frames. You made this copy in the `main()` function prior to calling this function.

Start the function by making a list of the contents of the current (*cropped*) folder ❷. The program will number each image sequentially, so use `enumerate()` with the `for` loop and set the `start` option to `1` ❸. If you haven't used `enumerate()` before, it's a handy built-in function that acts as an automatic counter; the count will be assigned to the `file_num` variable.

Next, name a variable, `img`, to hold the image and use the `open()` method to open the file ❹.

To fit the borders of a bounding box to Jupiter, you need all the non-Jupiter parts of an image to be black `(0, 0, 0)`. Unfortunately, there are stray, noise-related, nonblack pixels beyond Jupiter, and the edge of the planet is diffuse and gradational. These issues result in nonuniform box shapes, as shown in **Figure 15-5**. Fortunately, you can easily resolve these by converting the image to black and white. You can then use this converted image to determine the proper box dimensions for each color photo.

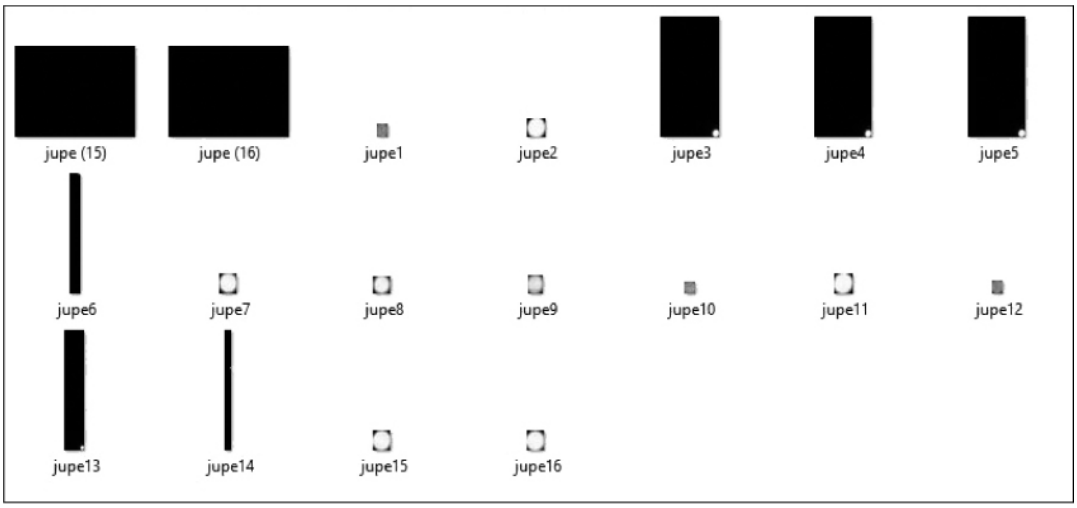

Figure 15-5: Irregularly sized cropped images due to problems defining the bounding box dimensions

To eliminate the noise effects that compromise the bounding-box technique, convert the loaded image to the "L" mode—consisting of 8-bit black and white pixels—and name the variable `gray`, for grayscale ❺. With this mode there is only one channel (versus the three channels for RGB color images), so you only need to decide on a single value when thresholding —that is, setting a limit above or below which an action occurs.

Assign a new variable, called `bw`, to hold a true black-and-white image ❻. Use the `point()` method, used to change pixel values, and a lambda function to set any value below 90 to black (`0`) and all other values to white (`255`). The threshold value was determined through trial and error. The `point()` method now returns a clean image for fitting the bounding box (**Figure 15-6**).

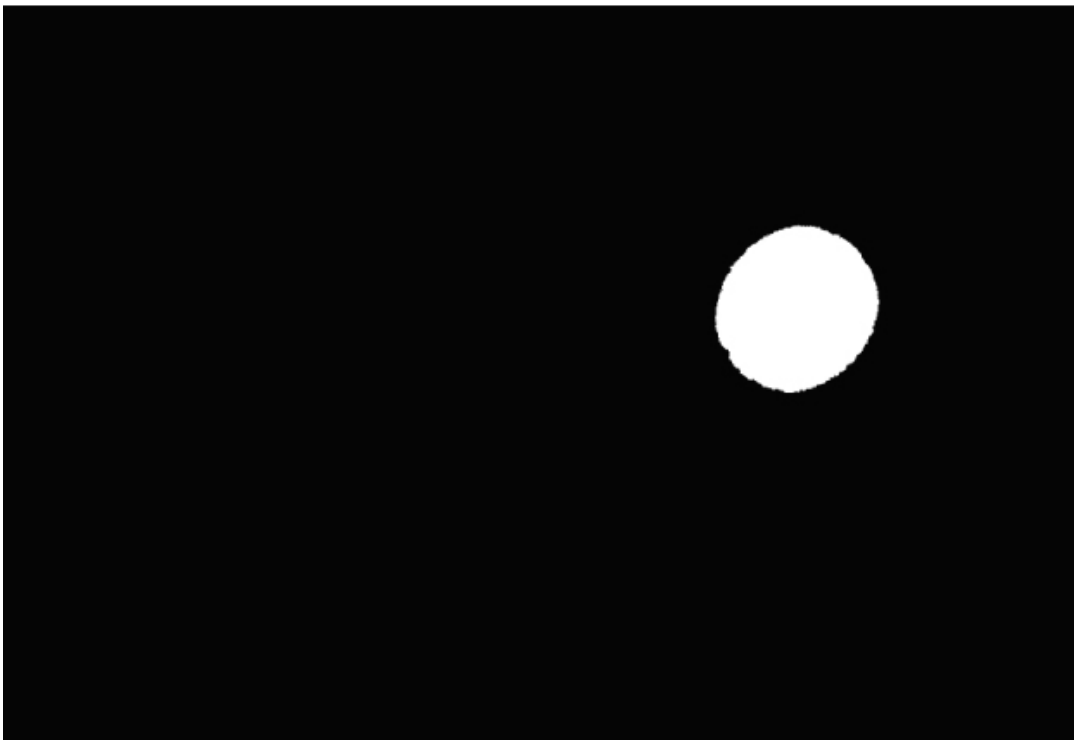

Figure 15-6: Screen capture of one of the original video frames converted to pure black and white

Now, call the `Image` module's `getbox()` method on `bw` ❼. This method prunes off black borders by fitting a bounding box to the nonzero regions of an image. It returns a tuple with the left, upper, right, and lower pixel coordinates of the box.

If you use `box` to crop the video frames, you get an image with borders tangent to Jupiter's surface (see the middle image in **Figure 15-7**). This is what you want, but it's not visually pleasing. So, add some black padding by assigning a new box variable, named `padded_box`, with its edges extended 20 pixels in all four directions (see the rightmost image in **Figure 15-7**). Because the padding is consistent and applied to all images, it doesn't compromise the results of cropping.

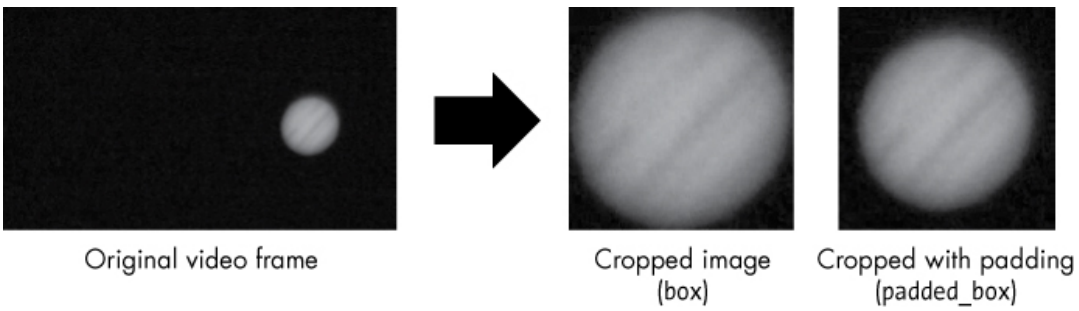

Figure 15-7: Initial crop tangent to Jupiter's surface (`box`) and final crop with padding (`padded_box`)

Continue by cropping each image with the `crop()` method ❽. This method takes `padded_box` as an argument.

To scale the image, use the `ImageOps.fit()` method. This takes the image, a size as a pixel width-and-height tuple, a resampling method, a border (`0` = no border), and even cropping from the center, designated by the tuple `(0.5, 0.5)`. The `pillow` module has several algorithm choices for resizing an image, but I chose the popular *Lanczos* filter. Enlarging an image tends to reduce its sharpness, but Lanczos can produce *ringing artifacts* along strong edges; this helps increase *perceived* sharpness. This unintended edge enhancement can help the eye focus on features of interest, which are faint and blurry in the original video frames.

After scaling, assign a `file_name` variable. Each of the 256 cropped images will start with *cropped_* and end with the number of the image, which you pass to the replacement field of the `format()` method. End the function by saving the file ❾.

Back in the global scope, add the code that lets the program run as a module or in stand-alone mode.

NOTE

I save the files using JPEG format because it is universally readable, handles gradations in color well, and takes up very little memory. JPEG uses "lossy" compression, however, which causes a tiny bit of image deterioration each time a file is saved; you can adjust the degree of compression at the expense of storage size. In most cases, when working with astrophotographs, you'll want to use one of the many lossless formats available, such as TIFF.

At this point in the workflow, you've cropped the original video frames down to a box around Jupiter; then you scaled the cropped images to a larger, consistent size (**Figure 15-8**).

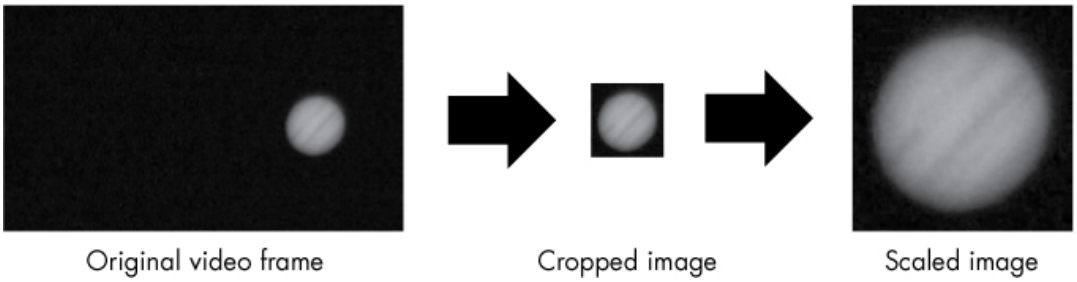

Figure 15-8: Relative sizes of images after cropping and scaling

In the next section, you write the code that stacks the cropped and scaled images.

The Stacking Code

The *stack_images.py* code takes the images produced by the last program and averages them so that a single stacked image is produced. You can download it from the book's resources at **https://www.nostarch.com/impracticalpython/**. Keep it in the same folder as the *crop_n_scale_images.py* program.

Listing 15-5 imports modules, loads images, creates lists of color channels (red, blue, green), averages the channels, recombines the channels, and creates and saves the final stacked image. It's simple enough that we won't bother with a `main()` function.

stack_images.py

```
❶ import os
   from PIL import Image

   print("\nstart stacking images...")

   # list images in directory
❷ os.chdir('cropped')
   images = os.listdir()

   # loop through images and extract RGB channels as separate lists
❸ red_data = []
   green_data = []
   blue_data = []
```

```
❹ for image in images:
        with Image.open(image) as img:
            if image == images[0]:  # get size of 1st cropped image
                img_size = img.size  # width-height tuple to use later
          ❺ red_data.append(list(img.getdata(0)))
            green_data.append(list(img.getdata(1)))
            blue_data.append(list(img.getdata(2)))

❻ ave_red = [round(sum(x) / len(red_data)) for x in zip(*red_data)]
   ave_blue = [round(sum(x) / len(blue_data)) for x in zip(*blue_data)]
   ave_green = [round(sum(x) / len(green_data)) for x in zip(*green_data)]

❼ merged_data = [(x) for x in zip(ave_red, ave_green, ave_blue)]
❽ stacked = Image.new('RGB', (img_size))
❾ stacked.putdata(merged_data)
   stacked.show()

❿ os.chdir('..')
   stacked.save('jupiter_stacked.tif', 'TIFF')
```

Listing 15-5: Splits out and averages color channels, then recombines into a single image

Start by repeating some of the imports you used in the previous program ❶. Next, change the current directory to the *cropped* folder, which contains the cropped and scaled images of Jupiter ❷, and immediately make a list of the images in the folder using `os.listdir()`.

With `pillow`, you can manipulate individual pixels or groups of pixels, and you can do this for individual color channels, such as red, blue, and green. To demonstrate this, you'll work on individual color channels to stack the images.

Create three empty lists to hold the RGB pixel data ❸, then start looping through the images list ❹. First, open the image. Then, get the width and height of the first image, in pixels, as a tuple. Remember, in the previous program, you scaled all the small cropped images to a larger size. You'll

need these dimensions later for creating the new stacked image, and `size` automatically retrieves this info for you.

Now use the `getdata()` method to get the pixel data for the selected image ❺. Pass the method the index of the color channel you want: `0` for red, `1` for green, and `2` for blue. Append the results to a data list, as appropriate. The data from each image will form a separate list in the data lists.

To average the values in each list, use list comprehension to sum the pixels in all the images and divide by the total number of images ❻. Note that you use `zip` with the splat (`*`) operator. Your `red_data` list, for example, is a list of lists, with each nested list representing one of the 256 image files. Using `zip` with `*` unpacks the contents of the lists so that the first pixel in image1 is summed with the first pixel in image2, and so on.

To merge the averaged color channels, use list comprehension with `zip` ❼. Next, create a new image, named `stacked`, using `Image.new()` ❽. Pass the method a color mode (`'RGB'`) and the `img_size` tuple containing the desired width and height of the image in pixels, which was obtained earlier from one of the cropped images.

Populate the new `stacked` image using the `putdata()` method and pass it the `merged_data` list ❾. This method copies data from a sequence object into an image, starting at the upper-left corner `(0, 0)`. Display the final image using the `show()` method. Finish by changing the folder to the parent directory and saving the image as a TIFF file named *jupiter_stacked.tif* ❿.

If you compare one of the original video frames to the final stacked image (*jupiter_stacked.tif*), as in **Figure 15-9**, you'll see a clear improvement in edge definition and the signal-to-noise ratio. This is best appreciated in color, so if you haven't run the program, take the time to download *Figure 15-9.pdf* from the website. When the image is viewed in color, the benefits of the stacking include smoother, "creamier" white bands, better-defined red bands, and a more obvious Great Red Spot. There is still room for improvement, however, so next you'll write a program to enhance the final stacked image.

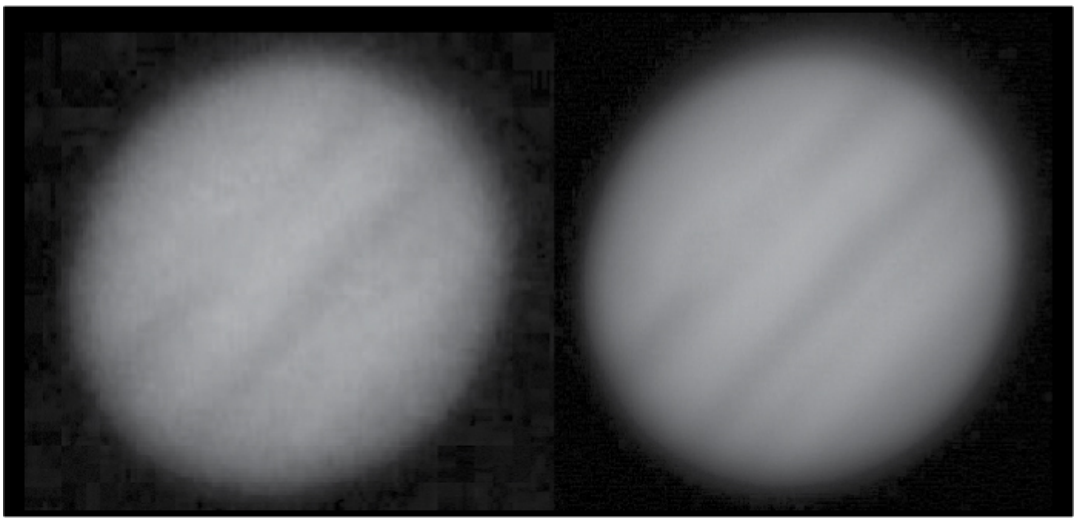

*Figure 15-9: An original video frame compared to final stacked image (*jupiter_stacked.tif*)*

NOTE

If the Great Red Spot looks pinkish to you in the stacked image, that's because it is! It fades from time to time, and many published pictures of Jupiter have exaggerated colors due to processing, so this subtle coloration gets lost. This is probably for the best, as "Great Pink Spot" just doesn't have the same ring to it.

The Enhancing Code

You've successfully stacked all the video frames, but Jupiter is still crooked, and its features are faint. You can further improve the stacked image using filters, enhancers, and transforms found in `pillow`. As you enhance images, you get further and further from the "ground truth" raw data. For this reason, I chose to isolate the enhancement process in a separate program.

In general, the first steps after stacking are to enhance details, using high-pass filters or an unsharp mask algorithm, and then to fine-tune brightness, contrast, and color. The code will use `pillow`'s image enhancement capability to apply these steps—though in a different order. You can download the code as *enhance_image.py* from **https://nostarch.com/impracticalpython/**. Keep it in the same folder as the previous Python programs.

NOTE

The processing of astronomical images can be quite involved, and whole books have been written on the subject. Some of the standard steps have been omitted in this workflow. For instance, the original video was not calibrated, and distortion effects due to turbulence were not corrected. Advanced software, such as RegiStax or AviStack, can prevent blurring by warping individual images so that distorted features, like the edges of cloud bands, overlap properly in all images.

Listing 15-6 imports `pillow` classes and opens, enhances, and saves the stacked image generated by the previous code. Because there are many possible options for enhancing images, I chose to modularize this program despite its small size.

enhance_image.py

```
❶ from PIL import Image, ImageFilter, ImageEnhance

❷ def main():
      """Get an image and enhance, show, and save it."""
   ❸ in_file = 'jupiter_stacked.tif'
      img = Image.open(in_file)
   ❹ img_enh = enhance_image(img)
      img_enh.show()
      img_enh.save('enhanced.tif', 'TIFF')

❺ def enhance_image(image):
      """Improve an image using pillow filters & transforms."""
   ❻ enhancer = ImageEnhance.Brightness(image)
   ❼ img_enh = enhancer.enhance(0.75)  # 0.75 looks good

   ❽ enhancer = ImageEnhance.Contrast(img_enh)
      img_enh = enhancer.enhance(1.6)
      enhancer = ImageEnhance.Color(img_enh)
      img_enh = enhancer.enhance(1.7)
```

```
    ❾ img_enh = img_enh.rotate(angle=133, expand=True)

    ❿ img_enh = img_enh.filter(ImageFilter.SHARPEN)

       return img_enh

if __name__ == '__main__':
    main()
```

Listing 15-6: Opens an image, enhances it, and saves it using a new name

The import is familiar except for the last two ❶. These new modules, `ImageFilter` and `ImageEnhance`, contain predefined filters and classes that can be used to alter images with blurring, sharpening, brightening, smoothing, and more (see **https://pillow.readthedocs.io/en/5.1.x/** for a full listing of what's in each module).

Start by defining the `main()` function ❷. Assign the stacked image to a variable named `in_file`, then pass it to `Image.open()` to open the file ❸. Next, call an `enhance_image()` function and pass it the image variable ❹. Show the enhanced image and then save it as a TIFF file, which results in no deterioration in image quality.

Now, define an enhancement function, `enhance_image()`, that takes an image as an argument ❺. To paraphrase the `pillow` documentation, all enhancement classes implement a common interface containing a single method, `enhance(factor)`, that returns an enhanced image. The `factor` parameter is a floating-point value controlling the enhancement. A value of `1.0` returns a copy of the original; lower values diminish color, brightness, contrast, and so on; and higher values increase these qualities.

To change the brightness of an image, you first create an instance of the `ImageEnhance` module's `Brightness` class, passing it the original image ❻. Mimic the `pillow` docs and name this object `enhancer`. To make the final, enhanced image, you call the object's `enhance()` method and pass it the `factor` argument ❼. In this case, you decrease brightness by 0.25. The `# 0.75` comment at the

end of the line is a useful way to experiment with different factors. Use this comment to store values you like; that way, you can remember and restore them if other test values don't yield pleasing results.

Continue enhancing the image, moving to contrast ❽. If you don't want to adjust the contrast manually, you can take a chance and use `pillow`'s automatic contrast method. First, import `ImageOps` from PIL. Then, replace the two lines starting with step ❽ with the single line: `img_enh = ImageOps.autocontrast(img_enh)`.

Next, punch up the color. This will help to make the Great Red Spot more visible.

No one wants to look at a tilted Jupiter, so transform the image by rotating it to a more "conventional" view, where the cloud bands are horizontal and the Great Red Spot is to the lower right. Call the `Image` module's `rotate()` method on the image and pass it an angle, measured counterclockwise in degrees, and have it automatically expand the output image to make it large enough to hold the entire rotated image ❾.

Now, sharpen the image. Even on high-quality images, sharpening may be needed to ameliorate the interpolation effects of converting data, resizing and rotating images, and so on. Although some astrophotography resources recommend placing it first, in most image-processing workflows, it comes last. This is because it is dependent on the final size of the image (viewing distance), as well as the media being used. Sharpening can also increase noise artifacts and is a "lossy" operation that can remove data—things you don't want to happen prior to other edits.

Sharpening is a little different from the previous enhancements, as you use the `ImageFilter` class. No intermediate step is needed; you can build the new image with a single line by calling the `filter()` method on the image object and passing it the predefined `SHARPEN` filter ❿. The `pillow` module has other filters that help define edges, such as `UnsharpMask` and `EDGE_ENHANCE`, but for this image, the results are indiscernible from `SHARPEN`.

Finish by returning the image and applying the code to run the program as a module or in stand-alone mode.

The final enhanced image is compared to a random video frame and the final stacked image in **Figure 15-10**. All the images have been rotated for ease of comparison.

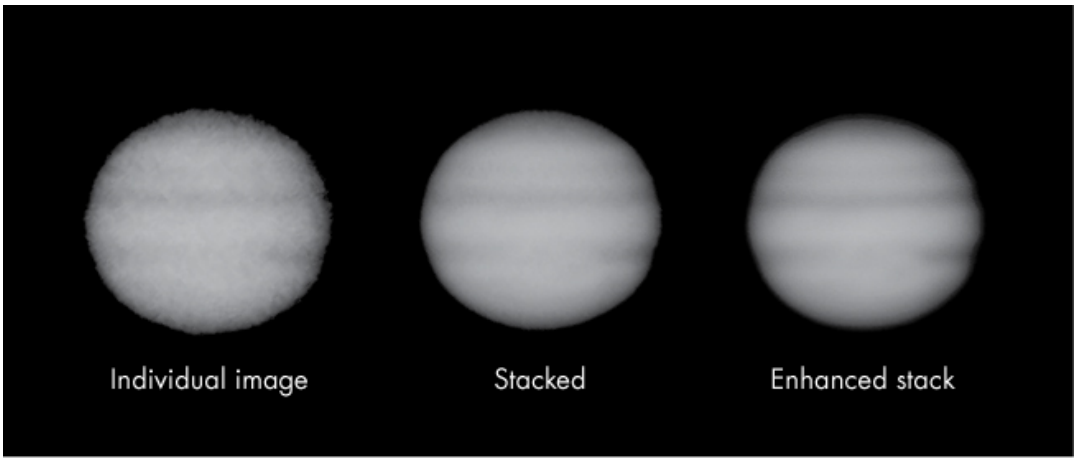

Figure 15-10: A random video frame, the results of stacking 256 frames, and the final enhanced image

You can see the improvement best when you view it in color. If you want to see a color version prior to running the program, view or download *Figure 15-10.pdf* from the website.

NOTE

If you're familiar with `pillow`*, you may be aware that you can use the* `Image.blend()` *method to stack images with only a few lines of code. To my eye, however, the resulting image is noticeably noisier than that obtained by breaking out and averaging the individual color channels, as you did with the* stack_images.py *program.*

Summary

The final image in **Figure 15-10** will never win any awards or be featured in *Sky & Telescope* magazine, but the point was to take on a challenge. And the result is a marked improvement over a single image captured from the video. The colors are brighter, the cloud bands sharper, and the Great Red Spot better defined. You can also make out the turbulent zone downwind of the Great Red Spot (refer to **Figure 15-1**).

Despite starting with rough input, you were able to register the images, remove noise through stacking, and enhance the final image using filters and transforms. And all these steps were accomplished with the freely available `pillow` fork of the Python Imaging Library. You also gained experience with the Python `shutil` and `os` modules, which you used to manipulate files and folders.

For more advanced image processing, you can use OpenSource Computer Vision (OpenCV), which you implement by installing and importing the `cv2` and `NumPy` modules. Other options involve `matplotlib`, `SciPy`, and `NumPy`. As always with Python, there's more than one way to skin a cat!

Further Reading

Automate the Boring Stuff with Python: Practical Programming for Total Beginners (No Starch Press, 2015) by Al Sweigart includes several useful chapters on working with files, folders, and `pillow`.

Online resources for using Python with astronomy include Python for Astronomers (**https://prappleizer.github.io/**) and Practical Python for Astronomers (**https://python4astronomers.github.io/**).

If you want to learn more about the OpenCV-Python library, you can find tutorials at **https://docs.opencv.org/3.4.2/d0/de3/tutorial_py_intro.html**. Note that knowledge of `NumPy` is a prerequisite for the tutorials and for writing optimized OpenCV code. Alternatively, SimpleCV lets you get started with computer vision and image manipulation with a smaller learning curve than OpenCV but only works with Python 2.

Astrophotography (Rocky Nook, 2014) by Thierry Legault is an indispensable resource for anyone interested in serious astrophotography. A comprehensive and readable reference, it covers all aspects of the subject, from equipment selection through image processing.

"Aligning Sun Images Using Python" (LabJG, 2013), a blog by James Gilbert, contains code for cropping the sun using the bounding-box technique. It also includes a clever method for realigning rotated images of the sun using sunspots as registration points. You can find it at

https://labjg.wordpress.com/2013/04/01/aligning-sun-images-using-python/.

A Google research team figured out how to use stacking to remove watermarks from images on stock photography websites and how the websites could better protect their property. You can read about it at **https://research.googleblog.com/2017/08/making-visible-watermarks-more-effective.html**.

Challenge Project: Vanishing Act

Image-stacking techniques can do more than just remove noise—they can remove anything that moves at a photo site, including people. Adobe Photoshop, for example, has a stack script that makes nonstationary objects magically vanish. It relies on a statistical average known as the *median*, which is simply the "middle" value in a list of numbers arranged from smallest to largest. The process requires multiple photos—preferably taken with a tripod-mounted camera—so that the objects you want to remove change positions from one image to the next, while the background remains constant. You typically need 10 to 30 pictures taken about 20 seconds apart, or similarly spaced frames extracted from a video.

With the mean, you sum numbers and divide by the total. With the median, you sort numbers and choose the middle value. In **Figure 15-11**, a row of five images is shown with the same pixel location outlined in each. In the fourth image, a blackbird has flown by and ruined the splendid white background. If you stack with the mean, the bird's presence lingers. But do a median stack on the images—that is, sort the red, green, and blue channels and take the middle values—and you get the background value for each channel (255). No trace of the bird remains.

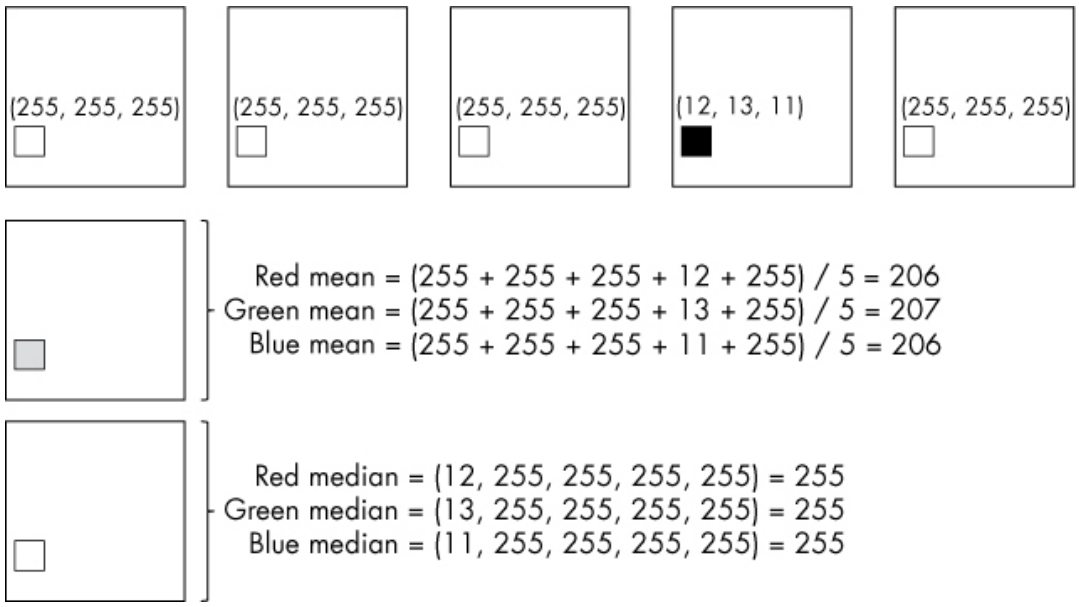

Figure 15-11: Five white images with the same pixel highlighted and its RGB values displayed. Median-stacking removes the black pixel.

When you average using the median, spurious values get pushed to the ends of the list. This makes it easy to remove outliers, such as satellites or airplanes in astrophotos, so long as the number of images containing the outlier is less than half the number of images.

Armed with this knowledge, write an image-stacking program that will remove unwanted tourists from your vacation happy snaps. For testing, you can download the *moon_cropped* folder from the website, which contains five synthetic images of the moon, each "ruined" by a passing plane (see **Figure 15-12**).

Figure 15-12: Synthetic moon photos for testing the median averaging approach

Your final stacked image should contain no evidence of the plane (**Figure 15-13**).

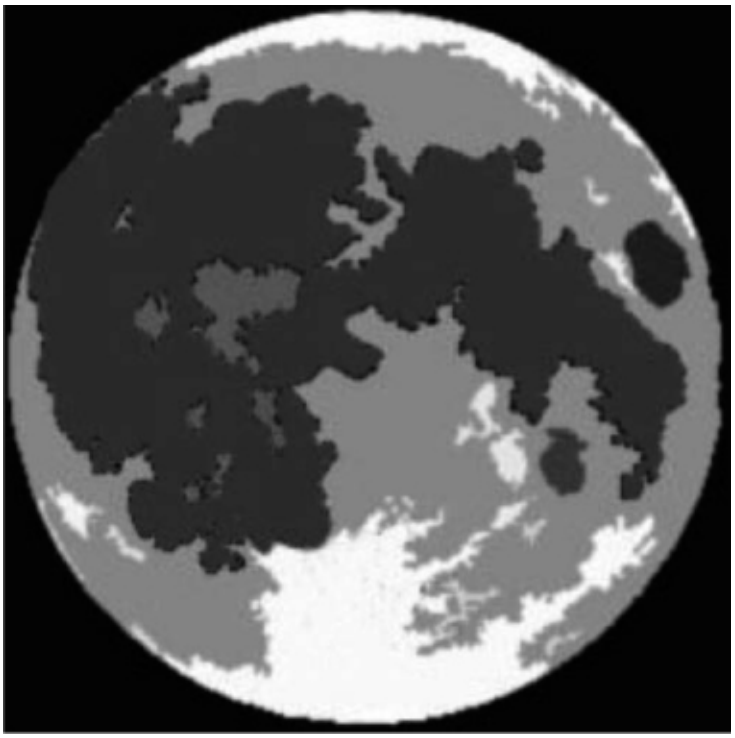

Figure 15-13: Result of stacking the images in the moon_cropped *folder using median averaging*

As this is a challenge project, no solution is provided.